

FIAP

PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Spec-Driven Development e Encerramento do Semestre

Aula 14/14 — Última aula · Sem entrega avaliada

Prof. Jorge Luiz Gomes

1h40min

A fronteira atual do código assistido por IA

Sem notebook — aula conceitual e demonstrativa

Agenda da Aula

- | | | |
|---|--|--------|
| 1 | Retrospectiva — a progressão completa do semestre | 8 min |
| 2 | O que é Spec-Driven Development — e por que surgiu agora | 12 min |
| 3 | SDD vs. Vibe Coding vs. Prompt Engineering direto | 10 min |
| 4 | O fluxo: spec → plano → tasks → implementação → revisão humana | 12 min |
| 5 | Ferramentas: GitHub Spec Kit, Claude Code, Kiro | 10 min |
| 6 | Demo ao vivo — de uma spec real ao código gerado | 12 min |
| 7 | Exercício — escreva sua primeira spec.md | 15 min |
| 8 | Próximos passos na carreira + CKPs como portfólio | 11 min |
| 9 | Discussão final + encerramento | 10 min |

OBJETIVO DA AULA

Encerrar o semestre com a fronteira mais atual do desenvolvimento assistido por IA. Não há entrega avaliada — a aula é conceitual e demonstrativa. O conteúdo crítico da disciplina foi todo coberto nas aulas 01–13.

Glossário

Spec-Driven Development (SDD)

Abordagem onde uma especificação estruturada — não o prompt solto nem o código direto — é o artefato central e a fonte de verdade do desenvolvimento assistido por agentes.

spec.md

Arquivo que documenta requisito, critérios de aceite e o que fica fora do escopo de uma feature — o ponto de partida do fluxo de Spec-Driven Development.

Tasks

Lista de tarefas atômicas e testáveis, geradas a partir do plano técnico, que o agente executa uma a uma.

Kiro

IDE spec-first da AWS onde a especificação vive lado a lado com o código, versionada, guiando a geração desde o design inicial.

Vibe coding

Pedir a um agente de código para implementar algo a partir de uma descrição solta, sem spec estruturada — rápido para protótipos, arriscado em produção pela falta de rastreabilidade.

plan.md

Decomposição técnica da spec: arquitetura, arquivos afetados e dependências — a etapa entre a spec e a lista de tasks.

GitHub Spec Kit

Projeto open-source que formaliza o workflow spec → plan → tasks, compatível com qualquer agente de código.

LangSmith

Plataforma de observabilidade da LangChain que registra cada chamada ao LLM, tool call e caminho de grafo percorrido — essencial para debugar agentes em produção.

01

Retrospectiva do semestre

Do primeiro chain de 5 linhas ao agente com StateGraph, checkpointing e HITL. A progressão não foi linear — foi deliberada.

A progressão do semestre — 14 aulas, 4 módulos

M1 — AULAS 01-04

LangChain LCEL

LCEL · ChatOllama · StrOutputParser ·
JsonOutputParser ·
ChatPromptTemplate · Memória
conversacional · Pydantic v2 ·
with_structured_output · Context
Engineering

CKP01 ✓ Chatbot profissional

M2 — AULAS 05-07

RAG

omic-embed-text · ChromaDB ·
cosine similarity · pipeline RAG
completo · chunking strategies ·
RAGAS · faithfulness · answer
relevancy · Gradio + Streamlit

CKP02 ✓ DocMind RAG

M3 — AULAS 09-11

Agentes ReAct

@tool decorator · AgentExecutor ·
ReAct loop ·
hub.pull("hwchase17/react") · context
rot · tiktoken · compressão de
observations · MCP overview ·
memória episódica

CKP03 ✓ Agente + Gradio público

M4 — AULAS 12-14

LangGraph

Router Chain · RunnableLambda ·
StateGraph · TypedDict ·
add_messages reducer · conditional
edges · draw_mermaid ·
MemorySaver · thread_id · HITL ·
interrupt_before · update_state

**Sem entrega — buffer de
segurança**

A LINHA VERMELHA DO SEMESTRE

Cada módulo resolveu a limitação do anterior. Chain não busca documentos → RAG. RAG não decide qual ferramenta usar → Agente. Agente não tem estado persistente nem pausa para humano → LangGraph. Cada passo foi motivado por um problema real.

Do primeiro prompt ao agente com HITL

- A01** **Revisão LCEL:** `prompt | llm | parser` — a pipe notation que une tudo. ChatOllama + ChatPromptTemplate + StrOutputParser em 5 linhas funcionais.
- A02** **Memória:** ConversationBufferMemory, WindowMemory, SummaryMemory. O chatbot passou a lembrar o que foi dito antes.
- A03** **Pydantic v2:** `with_structured_output()` + BaseModel. O LLM passou a retornar dados tipados, validados e fáceis de processar.
- A04** **Context Engineering:** window deslizante, system prompt eficiente, temperatura e parâmetros. O CKP01 entregou um chatbot profissional real.
- A05–07** **RAG completo:** `embed` → ChromaDB → `retrieve` → `augment` → `generate`. Chunking por token, RAGAS para avaliação. O chatbot passou a responder com base em documentos reais.
- A09–11** **Agentes ReAct:** `@tool` + AgentExecutor + loop Thought/Action/Observation. O sistema passou a *decidir* qual ferramenta usar em vez de seguir um fluxo fixo.
- A12–13** **LangGraph:** StateGraph + conditional edges + MemorySaver + HITL. O fluxo tornou-se explícito, auditável, persistente e com pausa para aprovação humana.

02

Spec-Driven Development — Fundamentos

A especificação estruturada — não o prompt solto, não o código direto — como artefato central do desenvolvimento assistido por agentes.

O que é Spec-Driven Development

Em 2025–2026, times de engenharia perceberam um padrão: pedir a um agente de código para "só fazer" (o chamado **vibe coding**) funciona para protótipos descartáveis, mas quebra em produção. Spec-Driven Development (SDD) inverte a lógica:

ANTES (VIBE CODING)

Humano descreve em linguagem solta → agente interpreta → gera código → humano descobre depois que o agente entendeu errado → retrabalho.

AGORA (SPEC-DRIVEN)

Humano escreve uma **spec estruturada** (requisitos, critérios de aceite, casos de borda) → agente planeja → agente implementa → humano revisa contra a spec, não contra a memória do que pediu.

A SPEC É A FONTE DE VERDADE

Em SDD, o código é derivável da spec — se o agente errar, você corrige a spec e regenera, em vez de remendar o código. Isso muda o que se revisa, o que se versiona e o que se discute em code review.

SDD vs. Vibe Coding vs. Prompt Engineering direto

ABORDAGEM	QUANDO USAR	RISCO PRINCIPAL
Vibe coding prompt solto, sem estrutura	Protótipo descartável, exploração rápida, script de uso único	Sem rastreabilidade da decisão · difícil revisar · não escala em time
Prompt engineering direto o que vocês praticaram o ano todo	Tarefas pontuais, análises, geração de conteúdo, scripts pequenos	Ainda depende do humano manter todo o contexto na cabeça
Spec-Driven Development a fronteira atual	Features de produto, código que precisa ser mantido, times trabalhando junto com agentes	Overhead de escrever a spec antes — mas paga-se com muito menos retrabalho depois

NÃO É SUBSTITUIÇÃO — É ESCALA

SDD não descarta o que vocês aprenderam sobre prompt engineering — a spec bem escrita é um exercício de context engineering aplicado a código. A diferença é o nível de estrutura e o fato de a spec virar um artefato persistente, revisável e versionado — não só uma mensagem de chat que desaparece.

O fluxo do Spec-Driven Development

- 1** **Escrever a spec** — o quê, não o como: requisitos, critérios de aceite, casos de borda, restrições. Em linguagem natural estruturada (spec.md).
- 2** **Plano técnico** — o agente (ou o humano) decompõe a spec em etapas técnicas: arquitetura, arquivos afetados, dependências.
- 3** **Tasks** — lista de tarefas atômicas e testáveis, geradas a partir do plano.
- 4** **Implementação pelo agente** — Claude Code, Copilot Workspace ou similar executa as tasks, editando arquivos reais do repositório.
- 5** **Revisão humana** — humano compara o resultado contra a spec (não contra a memória do pedido), aprova ou volta para o passo 1 com a spec refinada.

O LOOP É O PONTO

Quando a revisão reprova, o ajuste entra na **spec**, não direto no código gerado — assim a próxima geração já nasce corrigida, e a spec continua sendo a fonte de verdade auditável.

03

Spec-Driven na prática

Ferramentas reais, uma demo ao vivo e onde o mercado já adotou SDD em produção.

Ferramentas de Spec-Driven Development

GitHub Spec Kit

Open-source. Workflow `spec.md` → `plan.md` → `tasks.md`. Funciona com qualquer agente de código compatível. Gratuito.

Claude Code

Lê `CLAUDE.md` do repositório como spec persistente do projeto. Planeja, edita arquivos, roda testes, itera até passar.

Kiro (AWS)

IDE spec-first: a spec vive lado a lado com o código, versionada, e guia a geração desde o design inicial.

VOCÊS JÁ VIRAM ISSO FUNCIONANDO

Os arquivos `SOUL.md`, `CLAUDE.md` e `CLAUDE_NOTEB00KS.md` usados para gerar o material deste curso são exemplos reais de spec guiando um agente de código em produção — não é teoria, é como este próprio material foi construído.

FUNDAMENTAÇÃO — PLANNING MODE: A SEPARAÇÃO SPEC-EXECUÇÃO É TÉCNICAMENTE FORÇADA

Marco (*Agentic Coding with Claude Code*, Packt Publishing, 2025) detalha o mecanismo por trás do card do Claude Code acima: o modo `/plan` não é apenas uma convenção de processo — é uma restrição técnica. Durante o planning mode, o agente fica limitado a operações somente leitura (ler arquivos, analisar o código-base, consultar documentação) e não pode editar, criar ou deletar arquivos, fazer commit ou instalar pacotes até que o plano seja revisado e aprovado explicitamente. O autor conecta essa restrição ao motivo pelo qual SDD funciona na prática: uma especificação bem definida delimita o espaço do problema para o modelo, estabelece fronteiras claras e reduz alucinação, evitando efeitos colaterais como edições em arquivos fora do escopo. É o mesmo raciocínio por trás da linha "overhead de escrever a spec antes" da tabela do slide anterior — aqui com o mecanismo técnico que o sustenta.

Adoção no mercado

Iniciativas open-source

Um projeto open-source oficial (Spec Kit) formalizou o workflow spec → plan → tasks para uso com qualquer agente compatível.

Documentação de engenharia

Blogs de engenharia documentam como equipes internas usam agentes de código guiados por especificações persistentes (arquivos tipo CLAUDE.md) em repositórios de produção.

Ferramentas de IDE agêntica

Ferramentas comerciais de desenvolvimento agêntico nasceram de observar que times perdiam mais tempo corrigindo código mal especificado do que escrevendo specs completas de início.

No mercado de trabalho

"Escrever specs claras para agentes de código" já aparece como habilidade em vagas de engenharia — é a evolução natural do que vocês praticaram o ano todo em prompt e context engineering.

04

Próximos passos na carreira

Você aprendeu os primitivos. O que fazer agora? Trilhas concretas baseadas no que o mercado de 2026 está pedindo.

O que aprender depois deste semestre

TRILHA ENGENHARIA DE IA

Imediato (0-3 meses)

LangSmith para observabilidade · SqliteSaver/PostgresSaver para produção · FastAPI para servir agentes · Docker para containerizar

Médio prazo (3-6 meses)

LangGraph Platform · CrewAI ou Autogen · LoRA/QLoRA fine-tuning · Construção de MCP servers próprios

Longo prazo (6+ meses)

MLOps para agentes · Eval frameworks (RAGAS, Braintrust) · Contribuição open source · Publicação de papers ou blog técnico

RECURSOS RECOMENDADOS

DeepLearning.AI Short Courses

Cursos de 1–4h sobre LangChain, LangGraph, RAG, Agentes, Fine-tuning. Gratuitos ou barato. [learn.deeplearning.ai](https://www.deeplearning.ai)

LangChain Blog e Documentacao

Tutoriais oficiais sempre atualizados. blog.langchain.dev

Anthropic Engineering Blog

"Building Effective Agents" e outros artigos de produção. anthropic.com/engineering

Hugging Face Learn

Fine-tuning, PEFT, LoRA, datasets. huggingface.co/learn

LangSmith — observabilidade para agentes em produção

A maior dificuldade em produção não é construir o agente — é debugar *por que* ele tomou uma decisão errada. LangSmith resolve isso:

O QUE O LANGSMITH MOSTRA

- Cada chamada ao LLM: prompt exato, resposta, tokens, latencia
- Cada tool call: input e output de cada ferramenta
- Cada conditional edge: qual caminho foi tomado e por que
- Arvore de execução completa do grafo LangGraph
- Custo estimado por execução
- Comparacao A/B entre versoes de prompt

langsmith_setup.py

```
import os
from google.colab import userdata

# Habilitar LangSmith — basta setar env vars
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = userdata.get("LANGSMITH_KEY")
os.environ["LANGCHAIN_PROJECT"] = "agente-ckp03"

# A partir daqui: QUALQUER invoke() e tracado
# automaticamente. Zero mudanca no código.
resultado = agente.invoke({...})
# → trace disponivel em smith.langchain.com
# → ver todos os passos, tools, decisoes

# Free tier: 5.000 traces/mes — suficiente
# para desenvolvimento e demos
```

smith.langchain.com — free tier disponivel

Perguntas frequentes sobre carreira em IA

PERGUNTA	RESPOSTA HONESTA
"Preciso saber matemática avançada para trabalhar com IA?"	Para usar modelos (90% do mercado): não. Para pesquisa e treinamento de modelos do zero: sim (álgebra linear, cálculo, probabilidade). A maioria das vagas de engenharia de IA em 2026 exige saber usar e integrar modelos — não construir.
"Python é suficiente?"	Para IA generativa, sim. Para MLOps e infra: adicionar Docker, Kubernetes, cloud (AWS/GCP/Azure). Para data engineering que alimenta sistemas de IA: SQL é tão importante quanto Python.
"O LangChain vai ser substituído?"	Frameworks surgem e somem. O que não muda: entender os primitivos (chain, retriever, agent, graph). Quem entende os conceitos aprende o próximo framework em dias. Quem só decorou a API fica perdido quando o framework muda.
"Vale mais o portfólio ou o diploma?"	Os dois. O diploma abre a porta; o portfólio no GitHub demonstra o que você sabe fazer. Os CKPs deste semestre, bem documentados com README e exemplos rodando, já são um portfólio inicial sólido.

Seus CKPs como portfólio — como apresentar no GitHub

O QUE TODO RECRUTADOR OLHA NO GITHUB

README.md completo: O que faz, como rodar, print da tela funcionando

Demonstração rodando: Link do Gradio (share=True) ou GIF da tela

Código limpo: docstrings, comentarios, sem credenciais expostas

Estrutura de arquivos: não um notebook gigante, mas módulos organizados

```
README.md (estrutura recomendada)

# CKP03 – Agente Inteligente: [Dominio]

## Demo ao vivo
[[Open in Gradio](badge-url)](gradio-link)

## O que o agente faz
- Busca em documentos do dominio (RAG)
- Pesquisa na web (DuckDuckGo)
- Calcula métricas do dominio

## Stack
LangChain 0.3 | ChromaDB | Gradio | Ollama

## Como rodar no Colab
[[Open in Colab](colab-badge)](colab-link)

## Exemplos de uso
Q: "Quais os prazos do contrato?"
A: [screenshot da resposta]
```

05

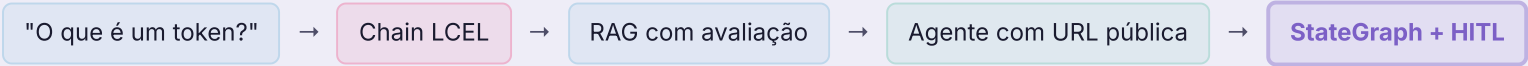
Encerramento do semestre

O arco completo de 2026, o stack que você domina, e o que fica depois que a última aula terminar.

O ano completo — 1º e 2º semestre juntos

SEMESTRE	FOCO	TECNOLOGIA CENTRAL	RESULTADO
1º semestre	Fundamentos de IA · Transformers · Prompt engineering · Fine-tuning conceitual · Hugging Face · Ética e LGPD	Python basics · Hugging Face Transformers · requests · gradio simples	Entender como LLMs funcionam e como interagir via prompt
2º semestre	LangChain LCEL · Memória · Pydantic · RAG · Agentes ReAct · LangGraph	LangChain 0.3 · ChromaDB · RAGAS · AgentExecutor · StateGraph · Gradio ChatInterface	Construir sistemas de IA completos, avaliáveis e com aprovação humana

Da teoria à produção — o arco completo



A linha do tempo da IA generativa — de onde vocês vieram

- | | |
|------|--|
| 2017 | Attention Is All You Need — Vaswani et al. O paper que criou a arquitetura transformer. Nada do que foi feito neste semestre existiria sem ele. |
| 2020 | GPT-3 — OpenAI. 175B parâmetros, few-shot learning. Mostrou que escala funciona. Base do paper "Language Models are Few-Shot Learners" (Brown et al.). |
| 2022 | ChatGPT — Interface conversacional para o grande público. 100M usuários em 2 meses. Prompt engineering vira habilidade valorizada no mercado. |
| 2023 | LangChain · GPT-4 · LLaMA — Frameworks de orquestração, modelos melhores, modelos open-source. A era das aplicações começa. |
| 2024 | Agentes · RAG maduro · MCP · LangGraph — Da cadeia ao agente. Sistemas que decide, busca, age e aprende com o resultado. O foco deste semestre. |
| 2026 | Deep Agents · Multiagentes · LangGraph Platform — Agentes em produção, operando por horas, com HITL, avaliação automatizada e orquestração multi-LLM. Vocês estão aqui. |

O stack que você domina ao final do semestre

LLM RUNTIME

ChatOllama via Ollama Cloud API · qwen3.6:27b · gpt-oss:120b · nomic-embed-text · temperature, top_p, num_predict

LCEL CHAINS

LangChain 0.3+ · pipe notation (|) · ChatPromptTemplate · StrOutputParser · JsonOutputParser · with_structured_output(Pydantic) · RunnableLambda · RunnablePassthrough

RAG STACK

ChromaDB + nomic-embed-text · RecursiveCharacterTextSplitter · as_retriever() · RAGAS (faithfulness, answer_relevancy, context_precision) · create_retriever_tool()

AGENTES REACT

AgentExecutor · @tool decorator · hub.pull("hwchase17/react") · verbose=True · max_iterations · DuckDuckGoSearchRun · tiktoken · compressão de observations

LANG GRAPH

StateGraph · TypedDict + Annotated[list, add_messages] · add_conditional_edges · MemorySaver · thread_id · interrupt_before · update_state · draw_mermaid() · invoke(None)

UI & DEPLOY

Gradio ChatInterface + streaming · share=True (URL pública) · Streamlit basics · Google Colab como ambiente de produção para demos

Debate final — perguntas para a turma

QUESTÃO 1 — REFLEXÃO PESSOAL

Qual foi o conceito mais surpreendente do semestre? Algo que mudou sua forma de enxergar IA?

QUESTÃO 2 — APLICAÇÃO PRÁTICA

Se você pudesse usar o que aprendeu em um problema real hoje — pessoal, familiar ou da cidade — qual seria?

QUESTÃO 3 — MERCADO

Você consegue imaginar uma empresa que ainda não usa agentes de IA mas que poderia se beneficiar muito? Qual problema resolveriam?

QUESTÃO 4 — ÉTICA

Qual limite você não daria a um agente autônomo, mesmo com HITL? Há ações que LLMs nunca deveriam poder iniciar?



Parabéns pela jornada

Em 14 aulas, vocês saíram de chains simples para sistemas com estado persistente, ferramentas reais, avaliação automática e aprovação humana antes de ações irreversíveis. Isso não é tutorial — é o que a indústria usa.

3

CKPs entregues

4

Módulos dominados

1

Agente público no ar

∞

Aplicações possíveis

Dúvidas, projetos futuros ou só para trocar ideia:

profjorge.gomes@fiap.com.br



Demo Prática ao Vivo

Escrever uma spec curta — requisito, critérios de aceite e o que fica fora do escopo — e usar um agente de código disponível gratuitamente para implementá-la, revisando o diff gerado contra a spec, não contra a memória do que foi pedido.

1. Escrever a spec

Requisito, critérios de aceite e o que fica fora do escopo — linguagem natural estruturada, sem código.

2. Agente implementa

O agente propõe um plano, gera o diff e explica as decisões tomadas a partir da spec.

3. Revisão contra a spec

A turma compara o resultado com os critérios de aceite originais — não com a lembrança do pedido.

STACK DA DEMO

Editor de texto simples para a spec · agente de código gratuito disponível (Claude Code ou equivalente) · repositório de exemplo já preparado

Escreva sua primeira spec.md

1. **Escolha uma ideia** — uma melhoria no seu agente do CKP03, uma feature nova ou qualquer problema pequeno que você queira resolver com código.
2. **Escreva o Requisito** em 2–3 frases: o que o sistema deve fazer, sem mencionar como implementar.
3. **Liste 3–4 Critérios de aceite** — condições objetivas que provam que a implementação está correta (formato "se X, então Y").
4. **Defina o que fica Fora do escopo** — o que deliberadamente não será tratado agora, para não inflar a spec.
5. **Troque com um colega** — ele tenta achar um caso de borda que sua spec não cobre. Essa é a revisão que, num fluxo real, aconteceria contra o código gerado.

💡 Não existe spec perfeita na primeira versão — o ganho está em achar a ambiguidade antes de um agente (ou colega) interpretá-la do jeito errado.

Referências

FERRAMENTA GitHub Spec Kit — github.com/github/spec-kit

BLOG Anthropic Engineering — Claude Code e desenvolvimento agentic. anthropic.com/engineering

FERRAMENTA Kiro (AWS) — IDE spec-first. kiro.dev

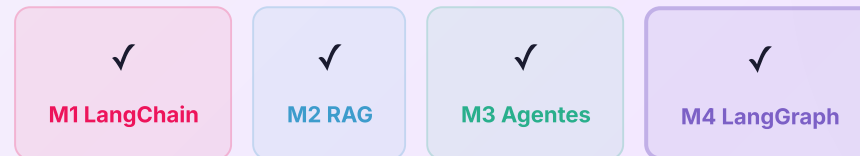
BLOG LangChain Blog e Documentação — tutoriais oficiais sempre atualizados. blog.langchain.dev

CURSO DeepLearning.AI Short Courses — LangChain, LangGraph, RAG, Agentes. learn.deeplearning.ai

CURSO Hugging Face Learn — fine-tuning, PEFT, LoRA, datasets. huggingface.co/learn

Semestre encerrado.

Vocês aprenderam a construir sistemas de IA reais. O que vem a seguir depende de vocês.



→ DEPOIS DESTA AULA

Não há próxima aula — o semestre está completo

Seus próximos passos: consolide os 3 CKPs como portfólio, escolha uma trilha de aprofundamento (observabilidade, multiagentes ou fine-tuning) e continue construindo. A jornada de Prompt & Context Engineering não termina aqui.